

Resenha Comparativa: Arquitetura de Microsserviços

Nessa resenha vamos analisar o artigo clássico de **Martin Fowler e James Lewis** e o capítulo 7 do livro **Engenharia de Software Moderna (ESM)**, do professor **Marco Tulio Valente**. Ambos os textos nos contam o que são microsserviços, no entanto, enquanto Fowler foca na filosofia e na forma como é usada nas empresas do Vale do Silício, Valente traz uma visão mais técnica e pé no chão sobre os desafios de engenharia, com foco na aplicabilidade acadêmica.

1. O Artigo de Fowler e Lewis: A Base de Tudo

O artigo do Fowler é o que podemos considerar um manual que popularizou o termo. Ele define microsserviços como uma forma de desenvolver uma aplicação única como um conjunto de pequenos serviços. Cada um roda seu próprio processo e se comunica com os outros através de APIs, geralmente HTTP.

O que mais chama a atenção no texto são as características que ele lista:

- **Equipes focadas em Negócio:** Ele diz que não faz sentido separar a equipe entre "pessoal do banco" e "pessoal do frontend". O certo é ter um time multidisciplinar que cuida de uma funcionalidade inteira (ex: o time de Pagamento).
- **Dados Descentralizados:** Esse é um dos pontos mais difíceis. Fowler defende que cada serviço deve ter seu próprio banco de dados. Se todo mundo usa o mesmo banco, o acoplamento continua lá e a agilidade morre.
- **Automação:** Ele deixa claro que se você não tiver uma cultura de entrega contínua (CI/CD) e automação de infraestrutura, tentar usar microsserviços vai ser um desastre.

O texto é muito otimista, mas Fowler termina com um aviso: microsserviços trazem para a aplicação um custo de complexidade alto. Ele sugere que, para muitos sistemas, um monólito bem feito ainda funciona melhor.

2. Engenharia de Software Moderna (Cap. 7): A Visão Técnica

O capítulo do professor Valente é mais estruturado para quem está estudando a disciplina. Ele começa explicando que microsserviços são, na verdade, uma forma extrema de **modularização**. A diferença é que, no monólito, a separação é lógica; nos microsserviços, a separação é física (em processos diferentes).

Pontos que o Valente aprofunda e que complementam o Fowler:

- **Problemas de Redes:** Diferente do Fowler, que foca na agilidade, Valente foca nos problemas. Ele lembra que chamadas de rede têm **latência** e

falham. No monólito, uma chamada de método é instantânea; no microsserviço, ela pode demorar ou cair.

- **Falhas Parciais:** Ele discute como o sistema deve se comportar quando um pedaço dele cai. É o conceito de "degradação graciosa": se o serviço de recomendação cair, o site de vendas tem que continuar funcionando, só que sem mostrar recomendações.
- **Escalabilidade:** Valente explica bem como é mais barato escalar microsserviços. Em vez de duplicar o sistema inteiro (monólito), você duplica apenas o serviço que está sobrecarregado.

3. Conclusão:

Lendo os dois textos, fica claro que microsserviços não são uma "receita de bolo" para qualquer projeto. Ao cruzar as ideias de Fowler e Valente, a conclusão lógica é que a migração para microsserviços não é uma decisão puramente técnica, mas **estratégica e econômica**.

Um ponto que se destaca é o que Fowler chama de "taxa de microsserviços" (*Microservice Premium*). Ele diz que microsserviços tornam o desenvolvimento mais lento no início devido à complexidade de configurar a rede, a segurança e a integração. O ganho de produtividade só aparece quando o sistema atinge um tamanho tão grande que um monólito se tornaria impossível de gerenciar.

Valente reforça essa ideia ao listar desafios que muitas vezes passam batidos por quem está empolgado com a tecnologia:

- **Dificuldade de Testes:** Testar uma funcionalidade que passa por cinco serviços diferentes é muito mais difícil do que testar um monólito.
- **Consistência de Dados:** No monólito, você tem transações ACID. Nos microsserviços, se um serviço de "Pedido" salva o dado mas o de "Pagamento" falha, você tem um problema de inconsistência que exige lógica complexa (como o padrão *Saga*) para resolver.
- **Observabilidade:** Como estudante, percebe-se que não basta "subir" o serviço; é preciso rastrear as requisições que saltam de um servidor para outro (Distributed Tracing), algo que Valente destaca como essencial.

Em conclusão, a lição principal é que **microsserviços servem para resolver problemas de escala (de software e de pessoas), não para simplificar o código**. Se o time é pequeno ou o sistema não precisa de deploys independentes constantes, o custo de manter essa arquitetura pode "quebrar" o projeto. Como bem pontuado no livro *Engenharia de Software Moderna*, a arquitetura deve ser "justa": nem simples demais que impeça o crescimento, nem complexa demais que atrepele a equipe.